

Plantilla Entrega 1. Proyecto Final

Bases de Datos No Relacionales

DermAssist: Sistema RAG NoSQL para enfermedades dermatológicas

Integrantes:

Blanca Lucy García García – 46143
Alejandra Montoya Rojas – 36751
Manuela Vallejo Toro – 28239

Universidad de Caldas

Docente:
Andrés Paolo Castaño

Octubre de 2026

ÍNDICE

Información del grupo **1**

1. Documento de análisis **1**
1.1. Universo del discurso . . . 1
1.2. Fuentes y volumen de datos . . . 2
1.3. Requerimientos funcionales . . . 2
1.4. Patrones de acceso . . . 3

2. Justificación del modelado NoSQL **4**
2.1. *Embedding* vs. *referencing* por relación . . . 4
2.2. Comparación general . . . 5

3. Diseño del esquema NoSQL **5**
3.1. Catálogo de colecciones . . . 5
3.2. Documentos de ejemplo . . . 7

4. Índices y *schema validation* **8**
4.1. Estrategia de índices . . . 8
4.2. Reglas de *schema validation* . . . 9

5. Configuración del entorno **10**

6. Plan de experimento de *chunking* **11**
6.1. Estrategias y parámetros . . . 11
6.2. Conjunto de 10 consultas de prueba . . . 12

Justificación de las 14 colecciones **12**

Organización del código **13**

Referencias de implementación **13**

Información del grupo

Campo	Información del grupo
Nombre del proyecto	DermAssist: Sistema RAG NoSQL para enfermedades dermatológicas
Dominio de datos	Enfermedades dermatológicas, síntomas, diagnósticos, tratamientos, medicamentos, imágenes clínicas y documentos de conocimiento
Modalidad	Terna
Integrante 1	Blanca Lucy García García · 46143 · blanca.garcia46143@ucaldas.edu.co
Integrante 2	Alejandra Montoya Rojas · 36751 · alejandra.montoya36751@ucaldas.edu.co
Integrante 3	Manuela Vallejo Toro · 28239 · manuela.vallejo28239@ucaldas.edu.co
Repositorio Git	https://github.com/Manu2480/Base-de-Datos-NoSQL
Infraestructura	MongoDB Atlas; la aplicación usa `MONGODB_URI` y la base `vector_db` por defecto
Lenguaje de la API	Python + FastAPI
Fecha de entrega	Octubre de 2026

1. Documento de análisis

1.1. Universo del discurso

DermAssist modela información sobre enfermedades, afecciones y lesiones dermatológicas. El sistema centraliza características clínicas, síntomas, manifestaciones visuales, tratamientos, medicamentos, contraindicaciones, historias médicas, resultados de laboratorio, imágenes y documentos de conocimiento. Se diferencia entre un **diagnóstico dermatológico establecido** —la descripción general de una afección— y un **diagnóstico asignado** —la ocurrencia de esa afección en un paciente y fecha determinados—.

El sistema está dirigido a usuarios académicos y de consulta informativa. Debe permitir buscar afecciones a partir de síntomas o texto libre, consultar tratamientos asociados, revisar riesgos de medicamentos según alergias, recuperar documentos y fragmentos relevantes, comparar imágenes de lesiones y generar respuestas en español con contexto y fuentes mediante RAG. No sustituye la valoración de un profesional de la salud y no debe operarse con datos reales de pacientes.

Los datos son heterogéneos y semiestructurados: hay atributos clínicos variables, arreglos de síntomas y medicamentos, documentos de texto, imágenes y resultados de laboratorio con estructuras propias. MongoDB es adecuado porque permite documentos flexibles, combinación de *embedding* y *referencing*, referencias entre colecciones, índices de texto y búsqueda vectorial en Atlas.

El código implementado está en `backend/`: `main.py` expone la API, `mongo.py` adminis-

tra la conexión y conversión BSON, `init_db.py` crea colecciones, validadores e índices, `chunking.py` implementa las estrategias y `ai.py` integra Jina y Groq.

1.2. Fuentes y volumen de datos

Cuadro 2: Fuentes y volumen de datos

Fuente o tipo de dato	**Uso en el sistema**	**Estado**
Documentos de conocimiento sobre dermatología	Texto para ingesta, fragmentación, *embeddings* y RAG	Implementado en `documentos_conocimiento`
Registros de diagnósticos, síntomas, medicamentos y tratamientos	Consultas clínicas de demostración y relaciones del dominio	Esquema Implementado *endpoints* Planeados-Definidos
Imágenes de lesiones	Almacenamiento en GridFS, metadatos y similitud visual	*Endpoint* Definidos; *dataset* final pendiente
Historias médicas y resultados de laboratorio	Consultas por paciente y trazabilidad clínica	Colecciones y *endpoints* Definidos
Embeddings generados por Jina	Búsqueda semántica de texto e imágenes	Requiere `JINA_API_KEY`

En la Entrega 2 se exige como mínimo 100 documentos de texto y 50 imágenes, dichos documentos e imagenes aun no se encuentran en el repositorio.

1.3. Requerimientos funcionales

Cuadro 3: Requerimientos funcionales

ID	**Requerimiento**	**Operación o componente MongoDB**	***Endpoint* implementado**
RF01	Búsqueda semántica sobre documentos fragmentados	`$vectorSearch` en `chunks` con `chunks_vector_index`	POST `/api/search` con `modo=vector`
RF02	Filtrar la búsqueda por idioma (pendiente de si se manejara español e ingles o solo español), fecha y estrategia	`$vectorSearch.filter` y filtros de aplicación	POST `/api/search`
RF03	Búsqueda de texto completo	Índice de texto; *fallback* controlado a expresión regular	POST `/api/search` con `modo=text`
RF04	Búsqueda de imágenes similares a una imagen	*Embedding* visual + `$vectorSearch` en `imagenes`	POST `/api/images/search`
RF05	Búsqueda de imágenes a partir de texto	*Embedding* textual y `$vectorSearch` visual	POST `/api/images/search-text`
RF06	Respuesta generada con contexto recuperado	Recuperación de *chunks* + Groq/Llama	POST `/api/rag`

Continúa en la siguiente página

Cuadro 3: Requerimientos funcionales (continuación)

ID	Requerimiento	Operación o componente MongoDB	*Endpoint* implementado
RF07	Consultar y registrar el historial de consultas de un usuario	Documento en `usuarios` con arreglo de consultas y/o `consultas`	GET/POST `/api/users/{user_id}/queries`
RF08	Consultar diagnósticos y tratamientos relacionados	Referencias y agregaciones entre colecciones	GET `/api/diagnoses`, GET `/api/diagnoses/{diagnosis_id}/treatments`
RF09	Consultar historia y secuencia de diagnósticos de un paciente	Referencias por `paciente_id` e `historia_medica_id`	GET `/api/patients/{patient_id}/history` y `/diagnosis-sequence`
RF10	Detectar riesgos de medicamentos considerando alergias	Lectura de pacientes, diagnósticos y medicamentos	GET `/api/patients/{patient_id}/medication-risks`
RF11	Ingerir documentos con una o varias estrategias de *chunking*	Inserción en `documentos_conocimiento` y `chunks`	POST `/api/documents` y POST `/api/ingest`

1.4. Patrones de acceso

Cuadro 4: Patrones de acceso

#	Patrón de acceso	Frecuencia	Lectura o escritura	Colecciones involucradas
1	Recuperar los *top-k chunks* por similitud, filtrando idioma, fecha o estrategia	Alta	Lectura	`chunks`, `documentos_conocimiento`
2	Buscar texto completo en *chunks* o documentos	Alta	Lectura	`chunks`, `documentos_conocimiento`
3	Recuperar contexto y generar una respuesta RAG con fuentes	Alta	Lectura	`chunks`, `documentos_conocimiento`, servicio Groq
4	Ingerir un documento, fragmentarlo y guardar *embeddings* por estrategia	Media	Escritura	`documentos_conocimiento`, `chunks`
5	Buscar una imagen similar o buscar imágenes mediante texto	Media	Lectura/ escritura	`imagenes`, GridFS
6	Consultar historia, secuencia diagnóstica y riesgos de medicamentos	Media	Lectura	`pacientes`, `historias_medicas`, `diagnosticos_asignados`, `medicamentos`, `tratamientos`

Continúa en la siguiente página

Cuadro 4: Patrones de acceso (continuación)

#	Patrón de acceso	Frecuencia	Lectura o escritura	Colecciones involucradas
7	Guardar la consulta de un usuario	Media	Escritura	`usuarios`, `consultas`

2. Justificación del modelado NoSQL

2.1. *Embedding* vs. *referencing* por relación

Cuadro 5: Estrategia de modelado por relación

Relación	Cardinalida	Estrategia	Patrón de acceso que la justifica	Riesgo considerado
Usuario → historial de consultas	1:N acotado	Híbrida: usuario con historial pequeño y colección `consultas` disponible	Consultar rápidamente el historial del usuario	El arreglo debe limitarse o archivarse para evitar crecimiento ilimitado
Documento → metadatos	1:1	*Embedded*	El nombre, idioma, tipo, fuente y fecha se recuperan con el texto	Bajo; son datos pequeños
Documento → imágenes	1:N	*Referenced*/híbrida	Las imágenes tienen ciclo de vida, *embeddings* y binarios propios	El binario se conserva en GridFS; no se incrusta en el documento
Documento → *chunks*	1:N	*Referenced*	La búsqueda vectorial opera sobre *chunks* independientes y cada estrategia genera sus propios registros	Muchos documentos y vectores de 1024 dimensiones
Diagnóstico → síntomas	N:M	*Referenced*	Un síntoma puede asociarse a varias afecciones	Requiere resolver referencias en consultas
Tratamiento → medicamentos	N:M	Híbrida	El tratamiento referencia el medicamento y embebe pautas generales acotadas	Cambios en medicamentos compartidos deben mantenerse consistentes
Paciente → alergias	1:N acotado	*Embedded*	Las alergias se consultan junto con el paciente para evaluar riesgos	Mantener la lista acotada y proteger datos clínicos
Paciente → historias médicas	1:N creciente	*Referenced*	Las historias y episodios crecen independientemente	Se requieren índices por `paciente_id`

Continúa en la siguiente página

Cuadro 5: Estrategia de modelado por relación (continuación)

Relación	Cardinalida	Estrategia	Patrón de acceso que la justifica	Riesgo considerado
Historia → imágenes y laboratorios	1:N	*Referenced*	Son recursos independientes y potencialmente grandes	Integridad de referencias y crecimiento

2.2. Comparación general

Cuadro 6: Comparación general entre *embedding* y *referencing*

Criterio	*Embedding*	*Referencing*	Decisión del grupo
Lecturas en una sola operación	Muy eficiente para datos dependientes y pequeños	Puede requerir consultas adicionales o `$lookup`	*Embedding* para metadatos, alergias y pautas; *referencing* para entidades independientes
Actualizaciones y consistencia	Simple en un documento; duplicar dificulta mantener consistencia	Centraliza entidades compartidas	*Referencing* para síntomas, medicamentos, diagnósticos e imágenes reutilizables
Tamaño y crecimiento	Adecuado para arreglos acotados; riesgo de documento grande	Maneja mejor crecimiento independiente	*Referencing* para historias, *chunks*, imágenes y laboratorios
Datos compartidos entre documentos	Puede duplicarlos	Permite reutilización	*Referencing*
Necesidad de `$lookup`	Menor	Puede ser necesaria	Se acepta en consultas de diagnóstico y recuperación de fuentes

3. Diseño del esquema NoSQL

3.1. Catálogo de colecciones

Cuadro 7: Catálogo de colecciones

Propósito	Estrategia	Referencias	Volumen estimado
`usuarios`			
Usuarios y datos de sesión/historial	*Embedded*/híbrida	`consultas`	Pendiente

Continúa en la siguiente página

Cuadro 7: Catálogo de colecciones (continuación)

Propósito	Estrategia	Referencias	Volumen estimado
`documentos_conocimiento`			
Fuentes textuales para el sistema RAG	Híbrida	`imagenes`, `chunks`	Mínimo esperado: 100
`imagenes`			
Metadatos y *embedding* visual de imágenes	*Referenced*; binario en GridFS	`documentos_conocimiento`, diagnósticos, pacientes	Mínimo esperado: 50
`chunks`			
Fragmentos vectorizados por estrategia	*Referenced*	`documentos_conocimiento`	Variable; varias estrategias por documento
`consultas`			
Consultas persistidas de usuarios	*Referenced*	`usuarios`	Pendiente
`evaluaciones`			
Resultados de evaluación RAG/*chunking*	*Referenced*	`consultas`	Opcional; se poblará al medir
`diagnosticos_dermatologicos`			
Catálogo de afecciones	*Referenced*	`sintomas`, `tratamientos`	Pendiente
`sintomas`			
Síntomas reutilizables	Independiente	Diagnósticos por ID	Pendiente
`medicamentos`			
Indicaciones, contraindicaciones e interacciones	Híbrida	Tratamientos y medicamentos relacionados	Pendiente
`tratamientos`			
Tratamientos de referencia	Híbrida	Diagnósticos y medicamentos	Pendiente
`diagnosticos_asignados`			
Diagnósticos de un paciente	Híbrida	`pacientes`, `historias_medicas`	Pendiente
`pacientes`			
Datos del paciente y alergias	*Embedded*/híbrida	Historias y diagnósticos	Pendiente; usar datos ficticios
`historias_medicas`			
Episodios clínicos	*Referenced*	Paciente, imágenes, laboratorios	Pendiente
`resultados_laboratorio`			
Resultados clínicos semiestructurados	*Referenced*	Paciente e historia	Pendiente

3.2. Documentos de ejemplo

Los siguientes ejemplos reflejan los nombres y campos usados por el código. Los vectores se abrevian en este documento; en MongoDB deben contener exactamente 1024 valores numéricos.

documentos_conocimiento

```
{
  "_id": "ObjectId(...)",
  "nombre_archivo": "guia_psoriasis_2026.txt",
  "tipo_documento": "Texto",
  "idioma": "es",
  "fuente": "Guía académica de dermatología",
  "contenido_texto": "La psoriasis es una enfermedad inflamatoria crónica de la
     piel caracterizada por placas ...",
  "fecha": "ISODate('2026-01-10T00:00:00Z')"
}
```

chunks

```
{
  "_id": "ObjectId(...)",
  "doc_id": "ObjectId(...)",
  "chunk_index": 0,
  "estrategia_chunking": "sentence-aware",
  "chunk_texto": "La psoriasis es una enfermedad inflamatoria crónica de la piel
     .",
  "embedding": [0.023, -0.117, 0.204, "... 1024 valores ..."],
  "modelo": "jina-clip-v2",
  "idioma": "es",
  "fecha": "ISODate('2026-01-10T00:00:00Z')",
  "fecha_ingesta": "ISODate('2026-03-16T00:00:00Z')"
}
```

imagenes

```
{
  "_id": "ObjectId(...)",
  "paciente_id": "ObjectId(...)",
  "historia_medica_id": "ObjectId(...)",
  "diagnostico_id": "ObjectId(...)",
  "archivo": "lesion_codo_001.jpg",
  "archivo_file_id": "ObjectId(...)",
  "descripcion_visual": {
    "colores": ["rojizo", "blanquecino"],
    "texturas": ["descamación"],
    "region_corporal": "codo"
  },
  "embedding_visual": [0.12, -0.08, 0.31, "... 1024 valores ..."],
  "modelo_embedding": "jina-clip-v2",
```

```
  "fecha_registro": "ISODate('2026-03-15T11:00:00Z')"
}
```

El *embedding* visual se calcula sobre el contenido binario de la imagen mediante Jina; no se calcula sobre la ruta o el nombre del archivo. El binario se guarda con GridFS.

diagnosticos_dermatologicos

```
{
  "_id": "ObjectId(...)",
  "nombre": "Psoriasis",
  "categoria": "Inflamatoria y autoinmune",
  "descripcion": "Afección inflamatoria crónica de la piel.",
  "sintomas_ids": ["ObjectId(...)"],
  "manifestaciones_visuales": ["Placas rojizas", "Escamas blanquecinas"],
  "tratamientos_ids": ["ObjectId(...)"]
}
```

pacientes y historias_medicas

```
{
  "_id": "ObjectId(...)",
  "codigo_paciente": "PAC-001",
  "nombre_completo": "Paciente ficticio",
  "fecha_nacimiento": "ISODate('1990-05-20T00:00:00Z')",
  "lugar_residencia": {"pais": "Colombia", "ciudad": "Manizales"},
  "alergias": [{"sustancia": "Penicilina", "reaccion": "Urticaria"}],
  "diagnosticos_asignados_ids": ["ObjectId(...)"],
  "historias_clinicas_ids": ["ObjectId(...)"]
}
```

```
{
  "_id": "ObjectId(...)",
  "paciente_id": "ObjectId(...)",
  "episodios": [{
    "fecha": "ISODate('2026-03-15T10:30:00Z')",
    "diagnosticos_asignados_ids": ["ObjectId(...)"],
    "observaciones": "Placas eritemato-descamativas.",
    "imagenes_ids": ["ObjectId(...)"],
    "resultados_laboratorio_ids": ["ObjectId(...)"]
  }]
}
```

4. Índices y *schema validation*

4.1. Estrategia de índices

Cuadro 8: Índices planificados e implementados por colección

Índice	Tipo	Campos	Consulta o patrón
`documentos_conocimiento`			
`idx_fecha_idioma`	Compuesto	`{ fecha: 1, idioma: 1 }`	Filtrar fuentes por fecha e idioma
`idx_documentos_texto`	Texto	`{ nombre_archivo: "text", contenido_texto: "text" }`	Búsqueda textual de documentos
`chunks`			
`idx_chunk_documento_estrategia_posicion`	Compuesto único	`{ doc_id: 1, estrategia_chunking: 1, chunk_index: 1 }`	Orden y no duplicación de *chunks*
`idx_chunks_texto`	Texto	`{ chunk_texto: "text" }`	Búsqueda textual y *fallback*
`chunks_vector_index`	Atlas Vector Search	`embedding`, 1024 dimensiones, `cosine`; filtros `idioma`, `estrategia_chunking`, `fecha`	Búsqueda semántica y RAG
`imagenes`			
`imagenes_vector_index`	Atlas Vector Search	`embedding_visual`, 1024 dimensiones, `cosine`	Similitud imagen-imagen y texto-imagen
`idx_imagen_diagnostico`	Simple	`{ diagnostico_id: 1 }`	Imágenes asociadas a diagnósticos
`pacientes`			
`idx_codigo_paciente`	Único disperso	`{ codigo_paciente: 1 }`	Identificar paciente
`historias_medicas`			
`idx_historia_paciente`	Simple	`{ paciente_id: 1 }`	Historia del paciente
`resultados_laboratorio`			
`idx_laboratorio_paciente_fecha`	Compuesto	`{ paciente_id: 1, fecha: -1 }`	Resultados recientes

La configuración usa `chunks_vector_index` e `imagenes_vector_index`, ambos con 1024 dimensiones y similitud `cosine`; la inicialización lo deja registrado en `init_db.py` y la documentación operativa está en `API_Y_PLANTILLA.md`.

4.2. Reglas de *schema validation*

La implementación aplica `validationLevel: "strict"` y `validationAction: "error"` para `documentos_conocimiento`, `chunks` e `imagenes`.

```
db.createCollection("documentos_conocimiento", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["nombre_archivo", "tipo_documento", "idioma", "contenido_texto",
       "fecha"],
```

```
    properties: {
      nombre_archivo: { bsonType: "string", minLength: 1 },
      tipo_documento: { bsonType: "string" },
      idioma: { bsonType: "string" },
      contenido_texto: { bsonType: "string", minLength: 1 },
      fecha: { bsonType: "date" }
    }
  }},
  validationLevel: "strict",
  validationAction: "error"
});
```

```
db.createCollection("chunks", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["doc_id", "chunk_index", "estrategia_chunking", "chunk_texto", "
        embedding", "modelo"],
    properties: {
      doc_id: { bsonType: "objectId" },
      chunk_index: { bsonType: ["int", "long"], minimum: 0 },
      estrategia_chunking: { enum: ["fixed-size", "sentence-aware", "semantic"]
          },
      chunk_texto: { bsonType: "string", minLength: 1 },
      embedding: {
        bsonType: "array", minItems: 1024, maxItems: 1024,
        items: { bsonType: ["double", "int", "long"] }
      },
      modelo: { bsonType: "string" }
    }
  }},
  validationLevel: "strict",
  validationAction: "error"
});
```

5. Configuración del entorno

Cuadro 9: Configuración del entorno

Elemento	Valor
Opción	MongoDB Atlas, cluster M0 según el informe; confirmar el *tier* actual
Base de datos	`vector_db` por defecto
Versión de MongoDB	MongoDB 8.3.11 2008R2Plus SSL
Región del cluster	AWS / N. Virginia (us-east-1) *Driver*
PyMongo `>=4.8,<5`	

Continúa en la siguiente página

Cuadro 9: Configuración del entorno (continuación)

Elemento	Valor
API	FastAPI `>=0.115,<1`, ejecutada con Uvicorn
Scripts de inicialización	`python -m backend.init_db`
Variables de entorno	`.env` local, basado en `.env.example`
Verificación de conexión	`admin.command("ping")`; `GET/api/health`
Embeddings	Jina `jina-clip-v2`, 1024 dimensiones; requiere `JINA_API_KEY`
LLM	Groq, por defecto `llama-3.1-8b-instant`; requiere `GROQ_API_KEY`
Almacenamiento de imágenes	GridFS

6. Plan de experimento de *chunking*

Las tres estrategias se aplican sobre los mismos documentos, con el mismo modelo de *embeddings*, recuperación, LLM, *prompt* y conjunto de consultas. Cada *chunk* almacena obligatoriamente `estrategia_chunking`.

6.1. Estrategias y parámetros

Cuadro 10: Estrategias de *chunking* y parámetros del código

Estrategia	¿Se implementará?	Parámetros del código	Librería/compone	Justificación
`fixed-size`	Sí	256 palabras por ventana, solapamiento de 32 palabras	`chunking.py`	Es una línea base simple para textos extensos
`sentence-aware`	Sí	Máximo 5 oraciones, solapamiento de 1 oración	Segmentación con expresión regular en `chunking.py`	Conserva oraciones completas y funciona con descripciones clínicas
`semantic`	Sí	Umbral por defecto 0.8; máximo 256 palabras; *embeddings* por oración	Similitud coseno implementada en `chunking.py` y *embeddings* Jina	Agrupa contenido relacionado y puede separar cambios de tema

Hipótesis: `sentence-aware` puede ofrecer el mejor equilibrio entre coherencia y costo para descripciones dermatológicas; `semantic` podría producir respuestas más relevantes en documentos con cambios temáticos, aunque con mayor costo de *embeddings*; `fixed-size` sirve como referencia y puede cortar listas o explicaciones.

6.2. Conjunto de 10 consultas de prueba

Cuadro 11: Consultas de prueba

ID	Consulta	Qué pone a prueba
C1	¿Qué enfermedades de la piel producen placas rojizas con escamas blanquecinas en codos y rodillas?	Relación entre síntomas y afección
C2	¿Cuál es el tratamiento tópico recomendado para la dermatitis atópica en niños?	Tratamiento y pauta pediátrica
C3	¿Por qué la epidermólisis ampollosa produce ampollas con traumatismos leves y cómo se cuidan las heridas?	Causa genética y cuidados
C4	¿En qué se diferencian el carcinoma basocelular y el carcinoma espinocelular?	Comparación de temas
C5	¿Qué características de una lesión pigmentada sugieren melanoma según la regla ABCDE?	Lista que puede fragmentarse
C6	¿Qué infecciones cutáneas se transmiten por contacto directo y cómo se tratan?	Información distribuida
C7	Una paciente alérgica a la penicilina tiene impétigo, ¿qué antibióticos debería evitar?	Contraindicaciones y alergias
C8	¿Cómo se distingue visualmente la rosácea del acné?	Comparación visual
C9	¿Qué factores desencadenan los brotes de lupus eritematoso cutáneo y cómo se maneja la enfermedad?	Causas y manejo
C10	Me salen manchas blancas en la cara y las manos que no duelen, ¿qué podría ser?	Lenguaje coloquial y búsqueda semántica

Métricas planeadas: número total de *chunks* por estrategia, longitud promedio del *chunk* y calidad percibida de la respuesta en escala de 1 a 5. Los resultados numéricos del experimento todavía no están incluidos en el repositorio ya que son Entrega 2.

Justificación de las 14 colecciones

Nuestro plan original definió 11 colecciones de dominio: `diagnosticos_dermatologicos`, `sintomas`, `medicamentos`, `tratamientos`, `diagnosticos_asignados`, `pacientes`,

`historias_medicas`, `imagenes`, `resultados_laboratorio`, `documentos_conocimiento` y `chunks`. El *backend* conserva esas 11 y agrega tres colecciones de soporte:

- `usuarios`: representa el actor que realiza consultas y permite guardar un historial acotado de las últimas 50 consultas. Es pertinente porque el requerimiento de historial necesita un propietario.
- `consultas`: queda disponible para una futura trazabilidad normalizada de consultas, respuestas, fuentes y tiempos de recuperación. No reemplaza el historial embebido: el arreglo embebido optimiza la lectura rápida y esta colección permite crecer sin inflar el documento del usuario.

- `evaluaciones`: permite guardar los resultados del experimento de *chunking* y de calidad RAG cuando se ejecute la segunda entrega. No es necesaria para la operación mínima actual y puede permanecer vacía hasta que existan mediciones.

No se necesita crear una clase o entidad Python por cada colección. MongoDB es documental y varias colecciones son catálogos o documentos semiestructurados. En este prototipo se usan modelos Pydantic solo para validar entradas HTTP, mientras MongoDB valida los documentos mediante *jsonSchema*. Crear 14 clases de dominio sin comportamiento propio duplicaría el esquema y no aportaría valor. Dicha implementación quedara más clara en la entrega II del proyecto.

Organización del código

```
backend/
  main.py # composición de la aplicación y rutas
  mongo.py # conexión y utilidades BSON
  init_db.py # colecciones, validadores e índices
  chunking.py # estrategias de fragmentación
  ai.py # embeddings y LLM
  services/file_storage.py # staging validado para Entrega 2
  tests/
frontend/static/ # HTML, JavaScript y CSS
data/incoming/ # archivos de preparación, no versionados
```

La separación por carpetas organiza responsabilidades técnicas; no obliga a crear una entidad por colección.

Referencias de implementación

- `README_IMPLEMENTACION.md`: instalación, ejecución, variables de entorno y limitaciones.
- `backend/main.py`: *endpoints* REST y validación de entradas.
- `backend/init_db.py`: colecciones, validadores e índices convencionales.
- `backend/chunking.py`: tres estrategias de fragmentación.
- `backend/ai.py`: *embeddings* Jina y generación con Groq.
- `backend/tests/test_api.py` y `backend/tests/test_chunking.py`: pruebas automatizadas.
- `docs/API_Y_PLANTILLA.md`: correspondencia entre requerimientos, *endpoints* e índices.